

# PolarOS — Our Approach, in Simple English

For the team | SIH26062 — Integrated Polar Expedition Logistics and Asset Management System (MoES/NCPOR)

Read this slowly, once, before you write any code or draw any screen. Everything we build follows from this note. If at any point you are confused about *why* we are doing something, come back here — the answer should be on this page. If it isn't, ask the mentor.

## 1. The problem, in three sentences

India sends an expedition to Antarctica once a year. Ships and planes can only reach the stations between December and February; for the other nine months, the 15–25 people living at Maitri and Bharati stations survive on whatever already arrived. Today this entire once-a-year supply chain runs on PDF forms, boxes marked with a pen ("Box 1 of 5"), paper registers, and the memory of a few experienced officers — so if a crate is missed, mislabelled, or runs out early, nobody sees it coming, and the fix is twelve months away.

The ministry has asked for one digital platform that covers five things: expedition planning, cargo tracking, inventory management, personnel movement, and emergency response.

## 2. The one special difficulty (and why normal software fails here)

Here is the trap in this problem: **Antarctica has almost no internet.** The stations have a satellite link so thin it is measured in kilobytes — think of a connection thousands of times slower than your phone. Sometimes there is no link at all for hours or days.

Almost every app you have ever used — Swiggy, IRCTC, Google Sheets — assumes the internet is always there. Take the internet away and they become blank screens. If we build a normal "cloud dashboard," it will work beautifully in the demo hall in Delhi and be useless at Maitri in July. The judges' evaluator is from NCPOR — they know this. So our whole design starts from one rule:

**Rule zero: every screen must work with zero internet. The internet, when it appears, is a bonus.**

This style is called **offline-first**. It is the opposite of how most software is built, and it is the reason our solution will look different from every other team's.

## 3. The core idea: a shared passbook, not a shared database

Think about a bank passbook. You never *erase* a line in a passbook. Every transaction — deposit, withdrawal — is a new line at the bottom. Your balance is not "stored" anywhere; it is simply what you get when you add up all the lines.

Our system works exactly like that. We keep **one table** called the **event log**. Every time something happens in the real world, we add one line — an *event* — to the log:

a crate is scanned at Cape Town → one event  
a station uses 20 barrels of diesel → one event  
a field party leaves the station → one event  
the party comes back → one event

We **never edit or delete** a line. If someone made a mistake ("we actually used 22 barrels, not 20"), we add a *correction event*. This is called **append-only** — you can only add to the end.

Then, whenever a screen needs to show something — "how much diesel is left?", "where is crate #214?", "who is outside the station right now?" — we don't look it up in some stored table. We **replay the log** and compute the answer, the way you compute a bank balance by adding up the passbook.

Why go to this trouble? Because of the internet problem. There are **three copies** of this log:

at **NCPOR headquarters in Goa** (full internet),  
on the **ship** (internet sometimes),  
at each **station** (almost never).

Each copy keeps working alone, adding its own events. When a connection appears — even for fifteen minutes — the copies exchange their new lines and catch up. And here is the beautiful part: because nobody ever *edits* anything, the copies can never fight with each other. Merging two passbooks is easy — you just combine the lines. Merging two edited spreadsheets is a nightmare — whose edit wins? By choosing append-only, we make the hardest problem in offline software (conflicts) simply *not exist*.

Two small tricks complete the picture:

**No clocks needed.** Each copy numbers its own events 1, 2, 3... So syncing is just: "Goa here — Maitri, I have your events up to number 4,821; send me everything after that." We never have to trust the clocks on machines that have been offline for months.

**Important things jump the queue.** Every event carries a priority. When the link opens for only fifteen minutes, "a field party is overdue" is sent before "we ate 3 kg of rice today." Safety first, paperwork later — literally.

That's the entire architecture. One table, three copies, add-only, replay for answers, priorities for the thin pipe. Everything else we build is just *screens on top of this log*.

## 4. The five modules — what each one actually does

The problem statement asks for five things. Each becomes one module, and each module is just a different way of reading and writing the same log.

**1. Expedition planning.** Before the season, Goa has ~400 items to send and three ways to send them: the ship (slow, huge, heavy cargo only), the small Basler aircraft (fast, but maximum ~1,800 kg *including the passengers*), and the ship's helicopter (only when the ship is nearby). We write a program — a *solver* — that takes every item's weight, size, danger class, and deadline, and assigns it to a vehicle without breaking any rule. Most importantly, if something *cannot* make it this season, the solver says so loudly in September — when there is still time to fix it — instead of the truth being discovered on the ice in January.

**2. Cargo tracking.** Every crate gets a QR sticker when it is packed in Goa. At every handover — Goa → Mumbai port → Cape Town → ship → station — someone scans it with a phone. Each scan is one event. So at any moment, anyone can ask "where is my crate and who touched it last?" and get the full chain. Dangerous items (fuel, batteries, chemicals) get a special flag and a packing checklist, because today those are handled by e-mail and can get rejected at the port.

**3. Inventory.** The station storekeeper spends two minutes a day recording what was used ("40 kg food, 18 barrels diesel"). From these small daily events, the system computes the thing nobody today can see: the **date each item will run out**, and whether that date falls *before* or *after* the next ship. If Maitri's diesel will hit zero on 2 October and the ship comes on 15 December, that is not a stock report — that is an alarm, ringing in March, when something can still be done. This one screen is the heart of our demo.

**4. Personnel movement.** One roster shows every member: their medical clearance, training, passport validity, and where they are right now (Goa / Cape Town / ship / station / out in the field). When a team goes out on the ice for research, they check out on the app — route, expected return time. If they are not back by then, the system raises the alarm *by itself*. Today, this depends on someone at the radio remembering.

**5. Emergency response.** All the emergency procedures (fire, medical evacuation, vehicle stuck in a storm) live inside the app, readable with no internet. And one special screen — the MEDEVAC card — answers the worst question in one tap: "*Can we evacuate a sick person right now?*" — by combining the nearest airstrip, whether flights are possible, and where the ship currently is.

Notice: all five modules read and write the **same log**. A scan, a diesel entry, and a check-out are all just events of different types. That is why a small student team can build five "systems" in one project — because it is really one system with five faces.

## 5. How we will prove it works (without going to Antarctica)

We obviously cannot test at Maitri. So we do three honest things instead:

**We generate a fake — but realistic — season.** A Python script creates one full expedition on the computer: 3 cargo lots on real-world dates, ~400 items, 35 members, daily consumption through the winter, thousands of events. We deliberately plant problems in it — a diesel shortage, an overdue field party — so our screens have something real to catch. Every number comes from NCPOR's own published documents (their forms, their notices, their flight limits), so the fake season behaves like a real one.

**We cut the cable in front of the judges.** Our demo includes a switch that throttles the connection down to Antarctic speed, or kills it entirely — live. The app keeps working; a small chip on screen says "OFFLINE — 214 events queued." Then we restore the link and the judges watch Goa's dashboard catch up. Seeing is believing.

**We measure instead of claiming.** How big is one day of station events? We actually compress it and weigh it in bytes, and show it fits inside a 15-minute satellite window with room to spare. A measurement beats an adjective.

## 6. What we are deliberately NOT doing (and you must be able to say why)

**No blockchain.** Blockchain solves the problem of people who *don't trust each other*. Goa, the ship and the stations all trust each other completely — their problem is *disconnection*, not distrust. Wrong tool.

**No AI chatbot.** Nobody at  $-30^{\circ}$  wants to chat with their inventory. They want one glance, one answer, one big button.

**No drones, no fancy hardware.** Our system needs only what already survives down there: an ordinary laptop or phone. If every computer but one dies, the surviving one carries the whole system — because the entire database is a single file that fits on a USB stick. Yes, walking a USB stick across the station is an officially supported way to sync. On the ice, that is not a joke; it is good engineering.

If a judge asks "why didn't you use X?", the answer is always the same shape: *we chose boring, unbreakable things on purpose, because the win condition of this problem is "still works when the cable is cut."*

## 7. The order we build in, and why

**Schema first** (the log's structure) — because everything sits on it.

**Fake season second** — because screens designed before data exist are fiction; screens designed after are previews.

**The three key queries** — diesel run-out date, crate trace, open parties — because these are the *answers* our screens will show.

**Six screens** — scan, stock, forecast, field parties, MEDEVAC, and Goa's dashboard. Mobile-first, big buttons (users wear gloves), every number on them taken from the queries, never invented.

**The demo story** — one three-minute walk: pack a barrel in Goa → scan at Cape Town → cut the link → log consumption → the forecast turns red → a party goes overdue → restore the link → Goa sees everything.

Data → answers → screens → story. Never the reverse order.

## 8. One promise we make to each other

Every number we show — on a slide, on a screen, in the demo — must be **real output of our own code**, and every fact about Antarctica must trace to **NCPOR's own published documents**. If our script produces 6,112 events, the slide says 6,112, even though an earlier draft said 5,439. If we don't know something, we say "we don't know yet" — judges forgive gaps; they never forgive bluffing. Our whole pitch is built on being the team whose every claim survives checking. Guard that.

*Next steps: the detailed worksheet (schema, generator, screens) and the prompt pack are separate documents. Read this note first; then the worksheet tells you what to build, and the prompt pack helps you build it.*